Lab 06 — Antwoorden met toelichting

Elk antwoord volgt hetzelfde schema: het antwoord, het mechanisme, de nuance of valkuil, en een voorbeeld dat je aan de terminal kunt nagaan.

Vraag 1 — "stop/start, mijn gegevens staan er nog"

Antwoord. Op één punt heeft hij gelijk: `stop` gevolgd door `start` bewaart de writable layer, dus ook de geschreven bestanden. De gegevens verdwijnen zodra de **container** verdwijnt: `podman rm`, `podman run --rm`, een `podman rm -f` gevolgd door een nieuwe `run` om een parameter te wijzigen of naar een nieuwe image over te stappen, een `podman-compose down`, of een orchestrator die de container vervangt.

Waarom. De writable layer hoort bij de container, niet bij de image of de engine. `stop` vernietigt niets: de container bestaat nog (`podman ps -a`), met zijn laag. `rm` verwijdert ze.

Nuance. De laag overleeft ook een herstart van de machine. Maar "de container nooit verwijderen" is geen strategie: bij de eerste image-update heb je een nieuwe container nodig, en dus een lege laag.

Voorbeeld.

```
podman run --name scribe alpine sh -c 'echo "notitie" > /data.txt'
podman start -a scribe && podman exec scribe cat /data.txt   # staat er nog
podman rm scribe && podman run --rm alpine ls /data.txt      # No such file or directory
```

Vraag 2 — De standaardpagina van nginx

Antwoord. `html` zonder `./` is geen pad: het is een **named volume** `html`, dat leeg en ter plekke aangemaakt wordt. Bij de eerste mount op `/usr/share/nginx/html`, dat in de image niet leeg is, **kopieert** de engine de inhoud van de image naar het volume (*copy-up*) — vandaar de welkomstpagina. Jouw map is nooit gemount. Correctie: `-v ./html:/usr/share/nginx/html:ro`.

Waarom. De `-v`-syntaxis onderscheidt bind mount en volume alleen aan de vorm van de bron: `/`, `./`, `~` → pad; anders → volumenaam. Het volume wordt zonder waarschuwing aangemaakt.

Nuance. `--mount type=bind,source=html,…` zou netjes gefaald zijn (pad bestaat niet) in plaats van stilletjes een volume te maken: dat is het belangrijkste argument voor de lange syntaxis.

Voorbeeld.

```
podman volume ls                        # er is een volume "html" bijgekomen
podman volume inspect html --format '{{.Mountpoint}}'
podman unshare ls $(podman volume inspect html --format '{{.Mountpoint}}')   # index.html,
50x.html: gekopieerd uit de image
podman rm -f -t 0 web && podman volume rm html
```

Vraag 3 — Permission denied op het volume

Antwoord. PostgreSQL draait in de container als UID 70 (`postgres`). In rootless wordt UID 70 van de container vertaald naar een UID uit je `subuid`-bereik: **100069** op een standaard Ubuntu/WSL. De bestanden zijn dus van 100069, en jouw gebruiker 1000 heeft er geen enkel recht op — dat is

de bedoeling. `podman unshare ls -ln …` lijst ze op, door de user namespace van Podman binnen te stappen.

Waarom. De user namespace koppelt alleen jouw UID aan root; alle andere UID's van de container wijzen naar het bereik `100000-165535`. De kernel past de permissies toe op de **echte** UID's.

Nuance. De bovenliggende map (`volumes/pgdata`) is wel degelijk van 1000: alleen wat de container schrijft, draagt een vertaald UID. `sudo ls` zou ook werken, maar dan gebruik je root om een gewilde isolatie te omzeilen.

Voorbeeld.

```
podman unshare cat /proc/self/uid_map
#          0       1000          1
#          1     100000      65536
podman unshare ls -ln ~/.local/share/containers/storage/volumes/pgdata/_data | head -3   # 70
70
ls -ln ~/.local/share/containers/storage/volumes/                                        #
1000 1000 pgdata
```

Vraag 4 — alpha of bravo?

Antwoord. **`alpha`**. De tweede container vindt in het volume een `PGDATA` die al geïnitialiseerd is en voert `initdb` niet opnieuw uit: `POSTGRES_PASSWORD=bravo` wordt genegeerd, het wachtwoord in de database blijft `alpha`.

Waarom. Het entrypoint-script van de officiële image maakt het cluster en de gebruiker alleen aan wanneer de map leeg is. De variabelen `POSTGRES_*` dienen enkel voor die initialisatie.

Nuance. Het wachtwoord wijzig je via SQL (`ALTER USER postgres PASSWORD 'bravo'`), of door het volume te verwijderen — mét de gegevens. Deze valkuil duikt op in CI wanneer een volume blijft rondslingeren tussen twee runs.

Voorbeeld.

```
podman logs db 2>&1 | grep -c initdb     # 0 bij de tweede start: niets geïnitialiseerd
podman exec db psql -U postgres -c "ALTER USER postgres PASSWORD 'bravo';"
```

Vraag 5 — Welk type voor welke behoefte

Antwoord. (a) **Bind mount**: je bewerkt de broncode op de host en de container ziet ze zonder rebuild. (b) **Named volume**: gegevens beheerd door de engine, correcte permissies, te back-uppen met `volume export`. (c) **Bind mount met** **`:ro`**: een bestand dat met het project meegeversioneerd wordt en dat de container niet mag wijzigen. (d) **tmpfs** als het RAM het toelaat, anders een eigen bind mount of volume — in geen geval de writable layer, die traag is en de container doet opzwellen.

Waarom. Het criterium is: wie moet de bestanden zien en beheren? Jij → bind mount; de engine → volume; niemand na het stoppen → tmpfs.

Nuance. Een tmpfs van 2 GB gebruikt 2 GB RAM, meegeteld in `--memory`. Voor (d) is een named volume dat de container zelf opruimt vaak realistischer.

Voorbeeld.

```
podman run --rm --tmpfs /tmp/pdf:size=2g alpine df -h /tmp/pdf
```

Vraag 6 — 1000 kan niet schrijven bij 1000

Antwoord. De twee "1000" zijn niet dezelfde gebruiker. De map is van **host**-UID 1000, en dat ziet de container als **root** (`0`). UID 1000 **van de container** is, gezien vanaf de host, 100999. Voor de kernel is dat een vreemde die in jouw map probeert te schrijven. Oplossingen: (1) `:U` op de mount, de engine doet een `chown` naar het UID van de container; (2) `--userns=keep-id`, jouw UID houdt nummer 1000 in de container, zodat het bestand echt van jou is.

Waarom. `uid_map`: `0 1000 1` en `1 100000 65536`. UID 1000 van de container valt in de tweede regel: `100000 + (1000 - 1) = 100999`.

Nuance. `:U` wijzigt de eigenaar **op de host**, blijvend (vraag 7). `keep-id` is bedoeld voor ontwikkeltools; het past niet bij images waarvan het proces onder een vast UID moet draaien (postgres = 70).

Voorbeeld.

```
podman run --rm --user 1000 -v ./uitvoer:/out:U alpine sh -c 'echo x > /out/f'     # OK
podman run --rm --userns=keep-id -v ./uitvoer:/out alpine sh -c 'id -u; echo y > /out/g'   #
1000, OK
```

Vraag 7 — Na `:U`

Antwoord. Op de host toont `ls -ln uitvoer` de map en de bestanden als eigendom van UID **100999** (UID 1000 van de container, vertaald), en zonder Podman kun je er niet meer in schrijven. Terugnemen doe je met `podman unshare chown -R 0:0 uitvoer` — "root" in de namespace, dat ben jij.

Waarom. `:U` is een echte recursieve `chown`, uitgevoerd door de engine in de user namespace. Hij wordt niet ongedaan gemaakt bij `rm` van de container.

Nuance. In de sandbox waarin deze labs geschreven zijn, begint het subuid-bereik bij 1001: daar verschijnt het bestand met UID 1001. Op jouw WSL is het 100999. Het mechanisme is hetzelfde, alleen het bereik verschilt (`cat /etc/subuid`).

Voorbeeld.

```
ls -ln uitvoer           # -rw-r--r-- 1 100999 1000 … f
podman unshare chown -R 0:0 uitvoer && ls -ln uitvoer    # 1000 1000
```

Vraag 8 — Veertien anonieme volumes

Antwoord. Ze komen van images die `VOLUME` declareren (postgres, mysql, redis, jenkins…) en gestart werden **zonder** `-v`: elke `run` maakt een anoniem volume aan, en `podman rm` verwijdert het niet. Per volume zegt `podman ps -a --filter volume=<naam>` welke container het gebruikt (geen → verweesd), en `podman volume inspect` geeft de aanmaakdatum. Voorkomen: volumes een naam geven en containers verwijderen met `rm -v`. `podman volume prune` verwijdert **alle** volumes

die door geen enkele bestaande container gebruikt worden — ook een named `pgdata` waarvan de container gisteren opnieuw aangemaakt werd.

Waarom. De engine kan "tijdelijk" niet van "kostbaar" onderscheiden; hij kent alleen "gemount door een bestaande container". Een named volume waarvan de container verwijderd is, is voor `prune` afval.

Nuance. `podman volume ls --filter dangling=true` lijst de volumes op die door geen enkele bestaande container gemount zijn, zonder iets te verwijderen — maar een named `pgdata` waarvan de container net verwijderd is, staat er ook tussen. Die lijst is om te **lezen**, niet om zomaar aan `rm` door te geven.

Voorbeeld.

```
podman image inspect postgres:16-alpine --format '{{.Config.Volumes}}'   # map[/var/lib/postgresql/data:{}]
podman volume ls --filter dangling=true
podman ps -a --filter volume=pgdata --format '{{.Names}}'
```

Vraag 9 — `tar` terwijl de database draait

Antwoord. Terwijl PostgreSQL schrijft, veranderen de gegevensbestanden en het journaal (WAL) voortdurend; `tar` leest ze na elkaar, op verschillende momenten. Het archief is een mengeling van toestanden: in het beste geval speelt PostgreSQL de WAL af en redt het zich, in het slechtste weigert het te starten (`invalid checkpoint record`). De twee correcte manieren: een **logische back-up** (`pg_dump`, consistent terwijl de database draait, overdraagbaar), of een **fysieke back-up met gestopte container** (`volume export`, `tar`). Voor dagelijks gebruik: `pg_dump`, dat geen onderbreking vraagt.

Waarom. `pg_dump` opent een transactie en leest een consistente momentopname van de database. Een bestandskopie kent dat begrip niet.

Nuance. Een back-up die nooit teruggezet is, bestaat niet: de terugzettest in een wegwerpcontainer hoort bij de procedure. Echte tools voor fysieke back-ups zonder onderbreking (`pg_basebackup`, pgBackRest) bestaan, maar vallen buiten dit lab.

Voorbeeld.

```
podman exec db pg_dump -U postgres > dump.sql          # tijdens het draaien, consistent
podman stop db && podman volume export pgdata -o pgdata.tar && podman start db   # fysiek, gestopt
```

Vraag 10 — `volume export` tegenover `tar` in een container

Antwoord. Het tweede commando start een wegwerpcontainer met Alpine en **twee** mounts: het volume `pgdata` op `/data`, alleen-lezen, en jouw huidige map op `/backup`; `tar` archiveert daar `/data` naar `/backup/pgdata.tgz`, dat dus bij jou verschijnt. Het heeft een container nodig omdat de bestanden van het volume, gezien vanaf de host, van UID 100069 zijn en jouw `tar` ze niet kan lezen — in de container zijn ze wél leesbaar. Deze methode werkt **ook met Docker**; `podman volume export` is eigen aan Podman.

Waarom. Een hulpcontainer is de universele manier om "een volume met de juiste rechten te zien". `podman volume export` doet intern hetzelfde, zonder container.

Nuance. `podman unshare tar czf … $(podman volume inspect pgdata --format '{{.Mountpoint}}')` is een derde weg, alleen voor Podman. In alle gevallen: container gestopt.

Voorbeeld.

```
podman run --rm -v pgdata:/data:ro -v ./:/backup alpine tar czf /backup/pgdata.tgz -C /data .
tar tzf pgdata.tgz | head -3
```

Vraag 11 — Een secret in tmpfs

Antwoord. De writable layer staat **op de schijf** van de host (`~/.local/share/containers/storage/overlay/…`): een secret laat daar een spoor na zolang de container bestaat, en mogelijk in een `podman commit`, een `podman export` of een schijfsnapshot. Een tmpfs zit **in het geheugen**: nooit naar schijf geschreven, gewist bij het stoppen, onbereikbaar via de opslag van de engine. De prijs: de grootte gaat van het RAM af en telt mee in `--memory`.

Waarom. tmpfs is een bestandssysteem van de kernel met het geheugen als drager (en de swap, als die aanstaat — vandaar het nut van `size=` beperken).

Nuance. Een tmpfs beschermt niet tegen een `podman exec` terwijl de container draait: het beschermt tegen **persistentie**, niet tegen rechtstreekse toegang.

Voorbeeld.

```
podman run --rm --tmpfs /run/secrets:size=1m alpine sh -c 'echo token > /run/secrets/t; df -h
/run/secrets'
```

Vraag 12 — Permission denied op RHEL

Antwoord. RHEL draait **SELinux** in *enforcing*-modus: de bestanden in `./conf` dragen het label van je homemap (`user_home_t`), en processen in containers mogen dat niet lezen, root of niet. Correctie: `-v ./conf:/etc/app:ro,Z`, dat de map herlabelt als `container_file_t`. Op Ubuntu/WSL is er geen SELinux (wel AppArmor): de optie wordt aanvaard en genegeerd. Ze opschrijven kost niets en maakt het commando overdraagbaar.

Waarom. SELinux voegt een **verplichte** toegangscontrole toe, los van UID's en `chmod`. `--user 0` verandert niets: de beslissing valt op basis van de labels.

Nuance. `:Z` labelt de map voor **deze** container; `:z` voor allemaal. Pas `:Z` nooit toe op een systeemmap (`/home`, `/etc`): het herlabelen breekt de toegang voor de rest van het systeem.

Voorbeeld.

```
podman run --rm -v ./conf:/etc/app:ro,Z alpine cat /etc/app/app.yaml   # RHEL: OK; WSL: ook OK
ls -Z conf   # RHEL: …container_file_t…; WSL: ? (geen label)
```

Vraag 13 — Twee API's, één volume `uploads`

Antwoord. Elke container mount **zijn eigen** lokale volume: een document dat bij replica 1 binnenkomt, is onzichtbaar voor replica 2, en het downloaden mislukt in de helft van de gevallen. Draaien de twee op verschillende machines, dan is het volume zelfs niet te delen. In het bedrijf gaan gebruikersbestanden naar **objectopslag** (S3, MinIO, Azure Blob), via HTTP bereikbaar vanuit

elke replica; de applicatie houdt **geen enkele lokale toestand** bij. Dat is de voorwaarde om een container wegwerpbaar te maken: je kunt er een doden, er drie starten, ze verplaatsen — er gaat niets verloren.

Waarom. Een volume is lokaal aan een host en een engine. Gedeelde toestand hoort in een dienst die daarvoor gemaakt is (database, objectopslag, netwerkcache).

Nuance. Met één replica op één machine werkt het volume — en net daardoor blijft de fout onzichtbaar tot de dag waarop je opschaalt. Kubernetes kent gedeelde volumes (`ReadWriteMany`, NFS), maar die blijven de uitzondering.

Voorbeeld.

```
podman run -d --name api1 -v uploads1:/app/uploads … ; podman run -d --name api2 -v
uploads2:/app/uploads …
podman exec api1 touch /app/uploads/doc.pdf ; podman exec api2 ls /app/uploads    # leeg
```